Data Management Plan

InProSSA – Industrial Problem Solving using Symbolic and Subsymbolic AI

Version 1.2 – 28 October 2026

Contents

Data Management Plan (DMP) **1**
Version History . . . 1
1. Purpose and Scope . . . 1
2. Roles and Responsibilities . . . 2
3. Data Inventory . . . 2
4. FAIR Principles . . . 5
5. Storage, Naming and Versioning . . . 6
6. Security and Sensitive Data . . . 6
7. Archiving, Publication and Preservation . . . 7
8. Legal and Ethical Aspects . . . 7
9. Responsibilities and Review . . . 7
Annex A — Repository Data Map . . . 7

Data Management Plan (DMP)

Project	InProSSA — Industrial Problem Solving using Symbolic and Subsymbolic AI
Funding	FFG, FO999923579
Work package	WP1 (Project Management)
Repository	https://github.com/RISCSoftware/inprossa
Version	1.2
Date	2026-10-28
Data Steward	Michael Bögl (michael.boegl@risc-software.at)

Version History

Date	Version	Change
01.07.2025	0.1	First version of Data Management Plan is available
31.01.2026	1.0	Benchmark generator added, synthetic benchmarks added
31.08.2026	1.1	Benchmark data from literature added
28.10.2026	1.2	roles and archiving consolidated; 2DPackLib dataset (D8) and experimental-branch data documented in §3.5

1. Purpose and Scope

This DMP describes how data generated and used in the InProSSA project is collected, documented, stored, shared, and preserved. It covers all work packages (WP1–WP6) and the shared repository at RISCSoftware/inprossa.

The project produces **no personal data**. All data is either

- **third-party benchmark data** (publicly available, re-used under its original terms),
- **synthetic/generated problem instances** (machine-generated, no personal reference), or
- **derived research data** (solver results, trained model weights, LLM interaction logs).

Scope exclusions: source code is versioned in Git and is not treated as research data here, although its releases are archived together with the data (see §7).

2. Roles and Responsibilities

Role	Person	Responsibility
Data Steward	**Michael Bögl**	Owns this DMP; maintains the data inventory; enforces naming, storage and archiving rules; approves data releases; single point of contact for data requests
Project lead / authors	Michael Bögl	Ensure data underlying each publication is deposited and referenced
All contributors	WP owners	Follow the conventions in §5, document provenance of new datasets, keep secrets out of the repository

The Data Steward reviews und updates this plan periodically.

3. Data Inventory

All paths are relative to the repository root.

3.1 Input data (used)

These datasets are publicly available and used within InProSSA according to the respective license. Repository paths for each dataset are listed in Annex A.

ID	Dataset	Origin / licence
D1	**CVRP benchmarks** (Golden, Christofides–Eilon) 33 instances (66 files), `.vrp` / `.sol`	CVRPLIB — sets E and Golden; re-used, not redistributed as own work
D2	**CVRPTW benchmarks** (Solomon) 112 files, `.txt` / `.sol`; classes C1/C2/R1/R2/RC1/RC2	Solomon benchmark, SINTEF TOP (Solomon 1987); public
D3	**2D bin-packing benchmarks** (Hopper & Turton, Wang, 2D-BPP CLASS) 20 JSON + 70+ instance dirs, `.txt` / `.json`	ESICUP Cutting & Packing datasets (`2d_rectangular`); public
D4	**JSSP benchmarks** (Taillard, Lawrence, Adams/Balasz/Zawack, Applegate/Cook, Storer/Wu/Vaccari) 155 files, `.txt`	JSPLIB, Taillard, jobshop.jjvh.nl; public
D5	**Reference quality table** 2 files, `.csv` / `.xlsx`	Compiled from JSPLIB `instances.json` and jobshop.jjvh.nl
D6	**Woodcutting use-case instances** (WC1–WC5) 18 files, `.txt` / `.pdf`	Project-internal use case (industrial partner); see §8
D7	**Woodcutting machine/simulator data** ~20 files, `.json` / `.txt`	Generated by project tooling
D8	**2DPackLib 2D packing instances** 512 files, `.ins2D`; CLASS (500, 10 classes × n = 20/40/60/80/100) and BENG (11)	2DPackLib (Univ. Bologna); public, re-obtainable from §3.4
D9	**AlphaZero / MuZero training logs** (WP5) ~15 files, ~2.7 MB, `.txt` / `.py` / `.tfevents`	Generated by project tooling (aborted experiment); branches `origin/msteindl2`, `origin/aborted/lightzero` only

ID	Dataset	Retention
G1	**OptDSL instances (hand-written)** 16 files, `.dsl`; 1DBP, 2DBP, CVRP, CVRPTW, JSSP, woodcutting	Preserve

ID	Dataset	Retention
G2	**OptDSL instances (converted)** ~300 files, ~25 MB, `.dsl`	Preserve (regenerable from D1–D4 + converters)
G3	**Synthetic 2DBP instances** 6 files, `.dsl`; n = 30/40/50	Preserve (regenerable via `Tools/generate_2dbp_instances.py`)
G4	**Synthetic woodcutting instances** ~10 files, `.json` / `.dsl` / `.mzn`	Preserve (regenerable via the problem data generator, §3.3)
G5	**Benchmark study results** 10 files, `.csv` / `.md`; MiniZinc vs. LNS quality, bound, runtime, gap	**Preserve — primary evidence for publications**
G6	**Instance-set definitions** 1 file, `.json`; Small/Medium sets	Preserve
G7	**LNS run artifacts** ~12 files, `.json` / `.dsl`; warm-start snapshots, trajectories, sessions	Preserve (selected runs)
G8	**LLM interaction logs** small, `.json` / `.jsonl`	Preserve (redacted, see §6)
G9	**Publications and reports** 16 PDFs, ~11 MB, `.tex` / `.pdf` / `.qmd` / `.bib`	Preserve
G10	**Documentation and figures** small, `.md` / `.svg` / `.png`	Preserve
G11	**Test and coverage artifacts** ~1 MB, `.xml` / `.html`	**Do not preserve** — reproducible build output
G12	**Environment and build definitions** small; `uv.lock`, `poetry.lock`, `Dockerfile`	Preserve — required for reproducibility
G13	**NL2DSL experiment results** ~290 files, `.json` / `.txt` / `.csv`	Do not preserve long-term — experimental branch, see §3.5

Total research data volume: ≈ 45 MB on `integration` (including duplicate copies of instances across work packages). The experimental branches in §3.5 hold a further ≈ 113 MB, which is not part of long-term preservation.

3.3 Synthetic data generation

Synthetic instances are **not** stored as an exhaustive collection. They are produced on demand by documented generators, and only the instances actually used in studies are kept in the repository. This keeps the data volume small while making every instance reproducible.

Generator	Produces	Location	Key parameters
Woodcutting problem data generator	Woodcutting instances (beams, boards, layers, faults) as JSON	`wp2/source/problem-data-generator/` (`main.py`)	`--beams`, `--beamlength`, `--layers`, `--boards`, `--board-length`, `--beamskipstart/end`, `--gap`, `--maxshiftcurvedcut`, `--f` (forbidden zones), `--randomseed`, `--defect_rate`, `--ratio_bad_curved`, `--bad-min/max-length`, `--curved-min/max-length`, `--compact`
Woodcutting DSL variant generator	OptDSL/MiniZinc variants of generated woodcutting instances	`wp5/source/unifying_-frame-work/Tools/generate_-woodcutting_dsl_vari-ants.py`, `convert_woodcutting_-benchmarks_noreorder.py`	source JSON, indexing base, reordering on/off
2DBP instance generator	Size-scaled 2D bin-packing instances	`wp5/source/unifying_-frame-work/Tools/generate_-2dbp_instances.py`	item count, box size, seed

Woodcutting synthetic data. The woodcutting use case is the project's industrial scenario, so real-world instances are limited (WC1–WC5, D6). To obtain statistically meaningful test sets, the **problem data generator** in `wp2/source/problem-data-generator/` creates arbitrary numbers of synthetic instances with

controlled difficulty: number of beams and boards, beam length, number of layers, minimum board length, gap and shift constraints, static forbidden zones, and the density and length distribution of bad and curved faults. Because the generator takes an explicit `--randomseed`, every synthetic instance is exactly reproducible from its parameter set.

Documentation requirement. Whenever a synthetic instance is used in a study or publication, the full generator invocation (all parameters, including the seed) must be recorded next to the result file or in the study README, so the instance can be regenerated without storing it.

3.4 Publicly available datasets used in this study

All third-party benchmark data used in the study is publicly available. The table lists the canonical download locations, so every instance in §3.1 can be re-obtained independently of this repository.

Problem class	Dataset / collection	Link	Used for
CVRP	CVRPLIB (PUC-Rio / UFF / UFPB)	https://galgos.inf.puc-rio.br/cvrplib/index.php/en/	Set E (Christofides–Eilon) and set Golden
CVRPTW	SINTEF TOP — Solomon benchmark	https://www.sintef.no/projectweb/top/vrptw/solomon-benchmark/	C1/C2/R1/R2/RC1/RC2 instances and best-known solutions
CVRPTW (extension)	SINTEF TOP — Gehring & Homberger benchmark	https://www.sintef.no/projectweb/top/vrptw/homberger-benchmark/	Larger VRPTW instances (optional extension)
2D bin packing / cutting & packing	ESICUP datasets (GitHub, CC0-1.0)	https://github.com/ESICUP/datasets	Hopper & Turton, Wang, 2D-BPP CLASS testset (`2d_rectangular`)
2D bin packing (NL2DSL)	2DPackLib (University of Bologna)	http://or.dei.unibo.it/library/2dpacklib	CLASS and BENG instance sets used in the NL2DSL study (D8)
JSSP	JSPLIB (GitHub)	https://github.com/tamy0612/JSPLIB	ta, la, abz, orb, swv instances and `instances.json` metadata
JSSP	Taillard benchmark page (HEIG-VD)	https://mistic.heig-vd.ch/taillard/problemes.dir/ordonnancement.dir/ordonnancement.html	Original Taillard instances
JSSP	jobshop.jjvh.nl	http://jobshop.jjvh.nl/	Best-known solutions and instance metadata

Availability note. The ESICUP data sets were migrated from `euro-online.org` to GitHub. Links were last verified on 2026-10-28.

3.5 Data on experimental branches

This data is documented here for completeness and provenance, but it is not part of the long-term preservation scope (§7). The one exception is the third-party 2DPackLib dataset (D8), which is a public benchmark and remains re-obtainable from its original source.

Branch	Unique data	Volume	Status
`origin/mctx`	2DPackLib instances (D8), NL2DSL results (G13)	885 files, 93 MB	Experimental; not merged
`Implementing_translation`	Earlier NL2DSL experiments and prompts; `FirstProject/*.mzn`	398 files, 17 MB	Experimental; superseded by `origin/mctx`
`origin/msteindl2`, `origin/aborted/lightzero`	AlphaZero/MuZero training logs (D9)	15 files, 2.7 MB	Aborted experiment
`main`, `problem-generator`, `0-based_DSL`, `origin/msteindl`	none	—	Contained in `integration`

Paths:

- `origin/mctx` — `wp2/source/NL2DSL/`
- `Implementing_translation` — `wp2/source/minizinc/NL2DSL/` (including `stable_-7.1.2026`, `examples_z3py`, `examples_optDSL`, `late_2025`) and `wp2/source/minizinc/FirstProject/`

- `origin/msteindl2`, `origin/aborted/lightzero` — `data_az/` and `data_mz_ptree/` at repository root

2DPackLib (D8). The 512 two-dimensional packing instances in `wp2/source/NL2DSL/problem_descriptions/2DPackLib/` are third-party benchmark data from the University of Bologna's 2DPackLib collection (§3.4). They are used as input for the NL2DSL experiments and are **not** project output. Because they are publicly available, they do not need to be preserved with the project: they can be re-obtained from the original source at any time. The repository copy is a convenience for reproducing the NL2DSL runs.

NL2DSL experiment results (G13). The `experiments/` and `prompts/` directories on `origin/mctx` hold LLM-generated OptDSL models, correctness results, solve-time/objective CSVs and prompt templates from the NL2DSL study. They are retained on the experimental branch for traceability but are not part of the long-term archive.

Handling. Experimental branches may be archived as tagged snapshots or deleted once their content is no longer needed. If any NL2DSL result is later used in a publication, the specific result files must be moved to `integration` and added to the preservation scope in §7 before the branch is removed.

4. FAIR Principles

4.1 Findable

- **Structured repository:** one directory per work package (`wp1`–`wp6`), with data separated from source code (`problem_instances/`, `output/`).
- **Descriptive file names:** problem class, instance family, size and variant are encoded in the file name (e.g. `2d_bpp_inst_cl_06_020_06.ins2D.dsl`, `jssp_10x5.dsl`, `Golden_1.vrp`).
- **Embedded metadata:** every generated `.dsl` file carries a header comment with its source, generator, indexing convention and key parameters; `instance_sets.json` documents the purpose, measurement environment and per-instance measured costs of the study sets.
- **README per dataset family:** `problem_instances/` READMEs, `studies/optdsl_bench/benchmarks/README.md`, `wp2/source/problem-data-generator/README.md`, `wp6/hybrid_solver/README.md`.
- **Persistent identifiers:** each public release is tagged in Git and deposited on Zenodo with a DOI; the DOI is cited in the corresponding publication and in this DMP.

4.2 Accessible

- **Internal access:** the Git repository (RISCSoftware/inprossa) is the single access point during the project; access is granted per project role.
- **Public access:** code and derived data are released under an open licence at project end (see §7). Third-party benchmarks are **not** re-published; instead the conversion scripts and source URLs are provided so they can be re-obtained from the original providers.
- **Open, non-proprietary formats:** plain text, JSON, CSV, Markdown, LaTeX — no vendor lock-in.
- **Third-party data stays retrievable:** every external dataset is referenced by its canonical public download link (§3.4), so access does not depend on this repository remaining online.
- **Authentication:** public data requires no authentication. Secrets (LLM API keys) are never part of the data and are excluded from the repository (`.gitignore`: `.env-secrets`).

4.3 Interoperable

- **OptDSL** is syntactically valid Python and is the single shared problem representation across all backends (MiniZinc, LNS, RL), which makes instances exchangeable between paradigms.
- **Standard community formats** are used for input data (TSPLIB/CVRPLIB `.vrp`/`.sol`, Solomon `.txt`, JSPLIB `.txt`, JSON for 2DBP), so instances remain usable with third-party tools.
- **MiniZinc `.mzn`** is emitted for the exact backend, enabling exchange with the wider CP community.
- **Encoding and units:** UTF-8 throughout; integer/discrete units as defined in the instance headers; CSV results use a fixed, documented column schema (`Class, Set, Instance, Size, MiniZinc Quality, MiniZinc Bound, MiniZinc Time (s), LNS Quality, LNS Time (s), LNS Stop, Gap (%)`).

4.4 Reusable

- **Provenance:** every converted instance is traceable to its source file and conversion script (`Tools/convert_*.py`, `converters/convert_*.py`); generated instances record their generator and seed (`Tools/generate_2dbp_instances.py`, `problem-data-generator --randomseed`).
- **Regenerable synthetic data:** synthetic woodcutting instances are reproducible from the documented generator invocation (see §3.3), so re-users can create additional instances of the same distribution instead of being limited to the stored sample.
- **Reproducibility:** pinned environments (`uv.lock`, `poetry.lock`, `Dockerfile`, `.devcontainer`), documented solver versions (cp-sat 9.15, MiniZinc 2.10.1, Python 3.11), explicit seeds and time budgets, and documented run commands in the study READMEs.
- **Documented semantics:** `architecture.md` files describe the LNS engine, operator contracts and data model; `docs/tooling.md` describes the analysis and synthesis tooling.
- **Licensing:** project code and derived data are released under an open licence; third-party data keeps its original licence and is referenced, not relicensed (see §8).
- **Quality assurance:** automated tests (`tests/`, `test/`) validate converters against published solutions, so re-users can verify that a converted instance preserves the original problem.

5. Storage, Naming and Versioning

Storage locations

Purpose	Location
Working copy / version control	Git repository RISCSoftware/inprossa
Large binaries (archives)	Git LFS or project file share, referenced from the repository
Long-term archive	Zenodo (DOI) + institutional backup
Local scratch	`.venv/`, `output/`, `htmlcov/` — excluded from version control

Naming conventions

- Instances: `<family>_<size>[_<variant>].<ext>` — e.g. `jssp_10x5.dsl`, `Golden_1.vrp`, `gen_bpp_n030_box100_s01.dsl`.
- Converted instances: original stem + `.dsl`, stored in a sibling `<family>_optdsl/` directory.
- Results: `<study>_<class>_<set>.{csv,md}` — e.g. `study_results_final__cvrp_medium.csv`.
- Run artifacts: `<instance>.<kind>.json` — e.g. `random_euclidean_cvrp_8.warmstart.json`.

Versioning

- All data under version control is versioned with Git; releases are tagged (`v<major>.<minor>`).
- Result files carry the generating command and solver/budget settings in their header or in the accompanying README, so a result set can be tied to a code revision.
- Superseded result sets are kept, not overwritten, and marked as superseded in the README.

Backup

- Repository: remote Git hosting with institutional backup.
- Non-versioned large artifacts: daily backup of the project file share.
- The Data Steward verifies restorability once per project year.

6. Security and Sensitive Data

- **No personal data** is collected or processed. No ethics approval or GDPR data-protection impact assessment is required.
- **Industrial use-case data (D6, D7):** woodcutting instances originate from the project's industrial use case. They contain no personal data but may be business-sensitive. They are kept in the internal repository only and are **not** part of any public release without written approval of the partner.
- **Secrets:** LLM API keys and endpoints are supplied via environment variables (`OPTDSL_AGENT_API_KEY`) or a per-user settings file outside the repository. Keys are masked in logs and session files (`api_key: "****"`). `.env-secrets` is git-ignored.

- **LLM logs (G8):** prompts and responses are stored for traceability. Before any release they are reviewed and redacted for keys, internal hostnames and partner-identifying content.
- **Committed credential (action required):** the branch `Implementing_translation` contains `wp2/source/minizinc/NL2DSL/.openrouter_api_key` with a live-looking OpenRouter key (`sk-or-v1-…`). The file is not present on `integration`, but it remains in the Git history and in that branch. **The key must be revoked and rotated**, and the file removed from history before the repository is made public. The Data Steward owns this action.

7. Archiving, Publication and Preservation

Aspect	Decision
What is published	OptDSL instances (G1–G4), study results (G5, G6), selected run artifacts (G7), publications (G9), documentation (G10), environment definitions (G12)
What is not published	Third-party benchmarks (D1–D5, D8, referenced only), industrial use-case data (D6, D7) without partner approval, build artifacts (G11), experimental-branch data (G13, D9)
Where	Zenodo (DOI per release) + public Git repository
When	At each publication and at project end
Metadata	`README.md` per dataset, `CITATION.cff`, `datasets.json` manifest listing every dataset with ID, description, format, size, licence and checksum
Retention period	Project duration + 10 years (institutional policy / FFG requirements)
Formats for preservation	Non-proprietary: `.dsl`, `.json`, `.csv`, `.md`, `.tex`, `.pdf/A` where feasible
Checksums	SHA-256 recorded in the release manifest

Known gap / action item: the repository `.gitignore` excludes `*.csv`, `*.zip`, `*.xls*` and `output/`. Result CSVs (G5) are therefore not automatically versioned. The Data Steward must explicitly force-add result files or move them to a tracked `results/` directory before each release. Target: next milestone.

8. Legal and Ethical Aspects

- **Third-party benchmark data** (CVRPLIB, Solomon, JSPLIB, Hopper/Wang, 2D-BPP CLASS) is used for scientific comparison under its original terms. It is not redistributed; instead, source references and conversion scripts are published so results can be reproduced. Canonical download links are listed in §3.4.
- **Project code and derived data** are released under an open-source licence (to be confirmed by the project lead; `liccheck` is configured in `pyproject.toml` to verify dependency licences).
- **Industrial partner data** is subject to the project consortium agreement; publication requires partner approval.
- **No ethical concerns** arise from the data itself. LLM-generated content (operators, constraints) is always validated by automated tests and human review before being used in reported results.

9. Responsibilities and Review

Task	Responsible	Frequency
Maintain data inventory (§3)	Data Steward	Each milestone
Verify FAIR compliance (§4)	Data Steward	Each release
Review secrets/redaction before release (§6)	Data Steward	Each release
Deposit release on Zenodo, assign DOI	Data Steward	Each publication / project end
Verify backup restorability	Data Steward	Yearly
Update this DMP	Data Steward	periodically

Annex A — Repository Data Map

Repository paths for every dataset in §3.1 and §3.2.

```
repository/
├── wp1/data_management_plan.md          this document
├── wp2/
│   ├── formulation/                     D6  use-case descriptions (PDF/TXT)
│   ├── documentation/                   G9/G10 reports, figures, bibliography
│   └── source/
│     ├── problem-data-generator/    G4       woodcutting synthetic instance generator (§3.
│      ├── simulator/               D7        machine states, simulation output
│      ├── minizinc/                  G2          MiniZinc-side models and data
│     ├── llm_backend/           G8       LLM client for bin-packing experiments
│      ├── llm_translator/           G2         LLM-assisted OptDSL translation
│      └── optimiser/                G2         incremental pipeline prototypes
├── wp3/
│   ├── examples/                        D6          WC1–WC5 use-case instances
│   └── deliverables/                    G9          D3.1, D3.2, OptDSL release
├── wp5/
│   └── source/unifying_framework/    D1–D5, G1–G7  OptDSL framework, instances, results
│     ├── problem_instances/         D1–D5, G1–G4  instances (input + converted)
│      ├── output/                      G5, G6, G7   study results, size sets
│      ├── studies/optdsl_bench/        G6           instance-set definitions
│      ├── publication/                G9         technical report, presentation
│      └── Tools/, converters/        provenance: conversion & generation scripts
└── wp6/
   ├── hybrid_solver/              G1–G7, G8   solver, instances, agent sessions
    └── final_report/                    G9          reporting templates

data_az/, data_mz_ptree/             D9        experimental branches only (§3.5)
```

Exact paths for the datasets whose location is not obvious from the tree:

ID	Path
D1	`wp5/source/unifying_framework/problem_-instances/cvrp/benchmarks/`
D2	`wp5/source/unifying_framework/problem_-instances/cvrptw/benchmarks/Solomon/`
D3	`wp5/source/unifying_framework/problem_-instances/2dbp/benchmarks/`
D4	`wp5/source/unifying_framework/problem_-instances/jssp/{ta,la,abz,orb,swv}/`
D5	`wp5/source/unifying_framework/problem_-instances/jssp/ReferenceQuality.*`
D8	`wp2/source/NL2DSL/problem_-descriptions/2DPackLib/`
G5	`wp5/source/unifying_framework/output/`, `.../publication/technical_report/results_-initial/`
G11	`coverage.xml`, `htmlcov/`, `python-junit.xml`